アルゴリズム基礎知識 II

第4章　キャンパスに図形を描こう

～ 配列を用いた最大値・最小値の探索とグラフィック応用 ～

章の概要 **本章の学習目標**

- **配列操作の習得:** 複数のデータ（座標やサイズ）を配列で効率的に管理する方法を学ぶ。
- **最大値・最小値アルゴリズム:** 配列の中から「最大の要素」「最小の要素」を見つけ出す基本構造を理解する。
- **キャンパス描写への応用:** 探索した最大値・最小値のデータを使い、画面（キャンパス）上に境界枠や特定の図形を描く実践力を身につける。

1. なぜ最大値・最小値の探索が必要か？

コンピュータグラフィックスやデータ処理において、ランダムに配置された多数の点（オブジェクト）の「外枠（バウンディングボックス）」を求めたり、最も大きい図形を強調表示したりする場面が頻繁に発生します。 これらの処理を実行するための基礎が**「配列要素の最大値・最小値探索」**です。



2. 配列から最大値を求める基本アルゴリズム

比較と更新を繰り替えす「トーナメント保持方式」

配列内の最大値を求める場合、人間のように全体を一度に見渡すことはできません。コンピュータは**「先頭の値を仮の最大値とし、1個ずつ比較して大きい方で更新する」**という手順を踏みます。

アルゴリズムの手順（最大値探索）

1. 配列の0番目の要素 `array[0]` を変数 `max_val` に代入する（初期化）。
2. ループを使って、1番目から最後の要素まで順番に比較する。
3. もし `array[i] > max_val` であれば、`max_val = array[i]` に更新する。
4. ループが終了した時点で、`max_val` に配列内の最大値が格納されている。

基本コード例（疑似コード / Pythonスタイル）

```
# 数値が格納された配列（キャンパス上の図形のX座標など） x_coords = [120, 450, 80, 320, 210] #
```

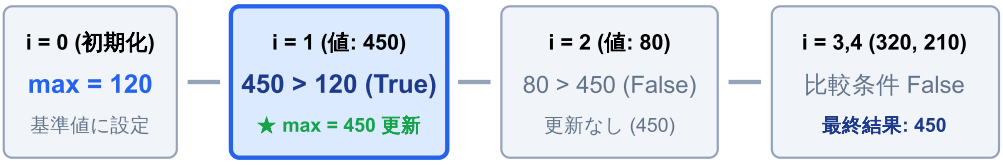
ポイント: 最小値を求めたい場合

最小値を求める場合は、比較演算子の向きを逆（ `array[i] < min_val` ）にするだけで全く同じロジックが使用できます。

3. トレース表で理解するアルゴリズムの動き

ループ実行時の変数変化（ドライラン）

配列 [120, 450, 80, 320, 210] に対して、最大値 max_x がどのように更新されていくかをステップ追跡（トレース）します。



トレース詳細表

ループ回数 (i)	配列の値 x_coords[i]	比較式 (x_coords[i] > max_x)	処理後の max_x
初期状態 (i=0)	120	-（初期化）	**120**
1回目 (i=1)	450	**450 > 120 → 真 (True)**	**450 (更新)**
2回目 (i=2)	80	80 > 450 → 偽 (False)	450 (保持)
3回目 (i=3)	320	320 > 450 → 偽 (False)	450 (保持)
4回目 (i=4)	210	210 > 450 → 偽 (False)	**450 (確定)**

4. 実践：キャンパス描写への応用アルゴリズム

複数図形をすっぽり包む枠を描画する

キャンパス上に複数の点や円が配置されているとき、それらすべての「最小X, 最大X, 最小Y, 最大Y」を探索することで、全体を囲む境界枠を自動描画できます。

境界枠（バウンディングボックス）計算プログラム

```
# キャンパス上の複数図形の中心座標リスト x_list = [100, 250, 400, 180, 320] y_list = [200,
```

解説 **なぜ幅と高さが計算できるのか？**

画面上の左端は「Xの最小値 (min_x)」、右端は「Xの最大値 (max_x)」です。
したがって、外枠の幅は max_x - min_x、高さは max_y - min_y で求めることができます。

5. 練習問題：理解度チェック

自力でアルゴリズムを組み立ててみよう

演習 1: 最も大きい円の半径を見つける

キャンパスに描かれた複数の円の半径データが配列 `radii = [15, 42, 8, 27, 35]` で与えられています。
最も大きい円の半径を求めて画面に表示するプログラムを作成しなさい。

> **ヒント:** 本章で学んだ `max_val` の探索アルゴリズムをそのまま当てはめてみましょう。

演習 2: 応用（チャレンジ問題）

配列 `scores = [65, 90, 45, 90, 72]` の中から最大値を求めるだけでなく、**「最大値が配列の何番目（インデックス）にあるか」**を保持するプログラムを考えてみましょう。

> **ヒント:** 値を保存する `max_val` に加えて、添字を保存する変数 `max_index = 0` を用意し、更新時に `max_index = i` とします。

まとめ

- 最大値・最小値の探索は「1つずつ比較して更新する」のが基本構造。
- 座標（X, Y）それぞれの最大・最小を求めると、図形を囲む境界枠が作れる。
- 値だけでなく「インデックス（位置）」を覚えることで、応用範囲が大きく広がる。